

**SUPPLY CHAIN PROCUREMENT ANALYTICS SYSTEM  
USING SQLITE, MACHINE LEARNING AND STREAMLIT**

**IT23723 - INTERNSHIP II REPORT**

Submitted by

**NAEHA S (241001142)**

in partial fulfilment for the award of the degree of

**BACHELOR OF TECHNOLOGY**

**IN**

**INFORMATION TECHNOLOGY**



**DEPARTMENT OF INFORMATION TECHNOLOGY  
RAJALAKSHMI ENGINEERING COLLEGE**

**JULY 2026**

## BONAFIDE CERTIFICATE

Certified that this project report titled SUPPLY CHAIN PROCUREMENT ANALYTICS SYSTEM USING SQLITE, MACHINE LEARNING AND STREAMLIT is the bonafide work of NAEHA S (24100142) who carried out the work during her 4 weeks Internship training. Certified further that based on the task given by the training organization, she has completed this project under their Supervision.

SIGNATURE OF THE REVIEWER

SIGNATURE OF THE MENTOR  
IN-CHARGE

SIGNATURE OF THE CLASS

SIGNATURE OF THE HOD

SIGNATURE OF THE ACADEMIC HEAD

**Dr. P. Valarmathie**

Head of the Department

Department of Information Technology

Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on \_\_\_\_\_

## ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our chairman **Mr. S. Meganathan**, B.E., F.I.E., for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan**, for her enthusiastic motivation which inspired us a lot in completing this project and Vice-Chairman **Mr. Abhay Shankar Meganathan**, B.E., M.S., for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college principal, **Dr. S. N. Murugesan**, M.E., Ph.D., for his kind support and facilities to complete our work on time. We extend heartfelt gratitude to **Dr. P. Valarmathie**, Professor and Head of the Department of Information Technology, for her guidance and encouragement throughout the work.

We extend our thanks to our project guide, faculty members, parents, and friends for their direct and indirect involvement in the successful completion of this project, for their encouragement and support.

NAEHA S

## TABLE OF CONTENTS

| S.NO | CONTENTS                                               | PAGE<br>NO |
|------|--------------------------------------------------------|------------|
| -    | About The Organization                                 | 7          |
| 1    | Abstract                                               | 8          |
| 2    | Introduction                                           | 9          |
| 3    | Literature Review and Background Study                 | 9–11       |
| 4    | System Requirements                                    | 11         |
| 5    | System Design and Methodology                          | 12–16      |
| 6    | Technology Stack                                       | 16         |
| 7    | Implementation                                         | 17–21      |
| 8    | Results and Discussion                                 | 22–24      |
| 9    | Challenges and Issues Faced                            | 25–27      |
| 10   | Testing                                                | 27         |
| 11   | Appendix A — Data Generation Script (generate_data.py) | 28         |
| 12   | Appendix B — KPI Engine (Full Listing)                 | 29         |
| 13   | Appendix C — requirements.txt                          | 29         |
| 14   | Appendix D — dashboard/app.py (Key Sections)           | 30–33      |
| 15   | Conclusion and Future Scope                            | 34         |
| 16   | System Usage / Use Case Description                    | 35–36      |
| 17   | User Manual — Running the Application                  | 36         |
| 18   | Glossary of Terms                                      | 37         |
| 19   | SDG Mapping                                            | 38         |

### **About the Organization – Merit Software Services Private Limited**

Merit Software Services Private Limited is a professional Information Technology and IT-Enabled Services (ITES) organization headquartered in Ambattur, Chennai, Tamil Nadu. Incorporated on 12 May 2004 under the Ministry of Corporate Affairs the company has built over two decades of expertise in business process outsourcing, data processing, call centre management, and software services. With operations spanning India and the United States and a workforce of approximately 82 professionals, Merit Software Services serves clients across both private and public sectors with a strong commitment to quality and customer satisfaction. The company is an ISO-certified organization and operates from a modern facility in the Ambattur IT corridor of Chennai.

The organization focuses on delivering comprehensive ITES solutions including end-to-end business process management, 24×7 inbound and outbound call centre services, high-accuracy data entry and data processing, application support, and IT-enabled business consulting. Through its call centre placement services vertical, Merit Software Services also provides talent acquisition and staffing solutions catered to the BPO and customer support industry. The company's service portfolio is designed to bridge organizational operational gaps through technology-driven, scalable, and cost-effective solutions for domestic and international clients.

During my internship at Merit Software Services, I was assigned to design and develop the ProcureSense AI – Procurement Intelligence Platform, an end-to-end data analytics and machine learning project. The project involved building a five-table relational SQLite data warehouse processing 28,765 purchase orders across 98 suppliers and 200 SKUs, authoring a portfolio of 10 production SQL queries using advanced window functions, CTEs, and Pareto analysis, engineering a six-model machine learning pipeline for late-delivery risk prediction with a champion Random Forest ROC-AUC of 0.714, and deploying a five-tab interactive Streamlit BI Studio with 10 Plotly charts and a live risk simulator. Through this hands-on project experience, I gained practical expertise in data engineering, relational database design, advanced SQL analytics, machine learning model development, SHAP explainability, and interactive business intelligence dashboard development. The internship significantly enhanced my technical capabilities, analytical problem-solving skills, and understanding of production-grade data and AI workflows in a professional IT-services environment.

## Abstract

ProcureSense AI is an end-to-end Enterprise Procurement Analytics and Business Intelligence Studio designed to demonstrate production-grade data engineering, SQL analytics, machine learning, and interactive dashboard skills within a single, cohesive platform. The system ingests and processes 28,765 purchase orders spanning January 2023 to December 2025 across 98 registered corporate suppliers and 200 technical SKUs totalling over ₹50,967 Crore in tracked procurement expenditure stored in a fully relational five-table SQLite database.

The project was developed in four major phases. In the first phase, a realistic multi-table procurement database was designed and populated using a stochastic data generation engine incorporating Ornstein-Uhlenbeck macroeconomic pressure signals, seasonal sine-wave modulation, and supplier-specific reliability drift. In the second phase, ten production-grade SQL portfolio queries were authored, covering window aggregations (SUM OVER, DENSE\_RANK, LAG), Common Table Expressions, Julian-day date arithmetic, and ABC Pareto spend classification. The third phase implemented a six-model machine learning pipeline — Naive Baseline, Supplier Heuristic, Logistic Regression, Random Forest Classifier, Tuned XGBoost, and Soft-Voting Ensemble — to predict late deliveries, achieving a best ROC-AUC of 0.714 with SHAP TreeSHAP explainability. The fourth phase delivered a five-tab interactive Streamlit BI Studio with ten Plotly visualisations, dark/light theme, live SQL editor, CSV export, and a real-time Late-Delivery Risk Simulator.

A composite Procurement Health Score (49.1/100) was engineered from five weighted KPI sub-components using Ordinary Least Squares price-inflation regression and Herfindahl-Hirschman Index spend concentration analysis. The platform identifies a 45.0% late delivery rate, ₹15,719 Crore in High-Risk supplier spend exposure, and a cost-optimal expected risk reduction of ₹87.90 million achievable through machine-learning-guided intervention. Results were benchmarked against the real-world Kaggle DataCo Smart Supply Chain dataset, confirming structural validity of the generated data patterns.

## 1. Introduction

Procurement and supply chain operations represent one of the most data-rich yet analytically under-served domains in enterprise business management. Large organisations routinely generate tens of thousands of purchase orders annually, each linking multiple dimensions — supplier identity, product category, delivery mode, pricing dynamics, and fulfilment reliability — yet meaningful cross-dimensional analysis is rarely performed in real time.

ProcureSense AI addresses this gap by combining modern data engineering, SQL analytics, and machine learning within a single, integrated platform. Rather than treating each of these disciplines in isolation, the system demonstrates how a well-designed relational database schema, a portfolio of production SQL queries, and a predictive ML pipeline can together produce board-ready KPI intelligence that translates directly into quantified financial risk reduction.

The platform processes a realistic 28,765-order dataset spanning three years (2023–2025) across 98 suppliers and 200 technical product SKUs. A stochastic data generation engine was designed to produce realistic procurement patterns — including supplier drift, commodity price pressure, seasonal demand variation, and macro-economic disruption signals. This data was then warehoused in a relational SQLite database with formal schema DDL, foreign key constraints, and multi-column performance indexes. Ten production SQL queries were authored to extract business insight, followed by a six-model machine learning pipeline for late-delivery risk prediction. The final phase integrated all outputs into an interactive Streamlit dashboard.

This report documents the complete development journey, including the specific technical decisions, modelling choices, and engineering challenges encountered at each stage.

## **2. Literature Review and Background Study**

### **2.1 Relational Database Design for Procurement Analytics**

The relational model, formalized by Codd (1970), remains the dominant paradigm for structured business data storage. For procurement applications, a normalized relational schema — separating suppliers, products, orders, deliveries, and inventory into distinct linked tables — is the standard approach in enterprise ERP systems such as SAP MM and Oracle Procurement Cloud. SQLite provides full SQL-92 compliance and supports the complex analytical queries (window functions, CTEs, subqueries) that modern procurement intelligence requires.

### **2.2 Window Functions and Advanced SQL for Business Analytics**

The SQL window function standard (ISO/IEC 9075-2:2003) provides OVER, PARTITION BY, ORDER BY, ROWS/RANGE BETWEEN, and ranking functions (DENSE\_RANK, ROW\_NUMBER, RANK) and lag/lead operators that enable complex analytical computations directly within a SQL SELECT statement. SQLite 3.25.0 (2018) introduced full window function support.

### **2.3 Machine Learning for Supply Chain Risk Prediction**

Predicting shipment delays at order creation time is an established, actively researched problem in operations research. Ensemble tree methods — Random Forests (Breiman, 2001) and gradient-boosted decision trees (Chen and Guestrin, 2016, XGBoost) — consistently outperform linear classifiers on tabular supply-chain data. Cost-sensitive threshold optimisation (Elkan, 2001) addresses the asymmetric cost structure inherent in procurement: a missed late delivery (stockout penalty) is typically an order of magnitude more costly than a false positive (unnecessary expediting cost).

### **2.4 SHAP Explainability for Procurement Decisions**

Shapley Additive exPlanations (SHAP), introduced by Lundberg and Lee (2017), provides a game-theoretic framework for explaining individual model predictions. TreeSHAP (Lundberg et al., 2018) computes exact SHAP values for tree ensemble models in polynomial time. In a procurement context, SHAP explanations allow a procurement manager to understand why a specific purchase order is flagged as high-risk — essential for building auditable, actionable AI decisions.

### **2.5 Procurement KPI Frameworks and Health Scoring**



Composite KPI scoring for supply chain health is widely used in enterprise procurement reporting. The Herfindahl-Hirschman Index (HHI) quantifies vendor spend concentration:  $HHI < 1,500$  indicates a healthy, competitive vendor market;  $HHI > 2,500$  signals dangerous concentration. Ordinary Least Squares price regression over multi-year purchasing data provides a statistically rigorous method for distinguishing structural commodity inflation from one-off price spikes.

## **2.6 Kaggle DataCo Supply Chain Dataset as Benchmark**

The DataCo Smart Supply Chain dataset (Constante et al., 2019, Kaggle) contains 180,519 real-world order records widely used as a benchmark for supply chain ML experiments. This project integrates the DataCo dataset as an external validation benchmark, confirming that the structural patterns in the synthetically generated procurement data are consistent with real-world supply chain operations.

## **2.7 Summary**

The background review confirms four things that directly shaped this project's design: (1) a normalized five-table schema is the appropriate foundation; (2) SQL window functions are the most efficient mechanism for business KPI computation; (3) ensemble tree methods with cost-sensitive threshold optimisation are appropriate for late-delivery prediction; and (4) SHAP explainability is necessary to make ML-driven decisions auditable.

# **3. System Requirements**

## **3.1 Hardware Requirements**

- Development machine with at least 8 GB RAM
- Modern CPU with 4+ cores for parallel data generation and XGBoost training
- Internet connectivity for Kaggle dataset download
- At least 2 GB free disk space for database, model checkpoints, and artefacts

## **3.2 Software Requirements**

- Python 3.10 or later
- pandas  $\geq 2.0$  and NumPy (data loading, transformation, KPI computation)
- scikit-learn (Logistic Regression, Random Forest, preprocessing, evaluation)
- XGBoost  $\geq 1.7$  (Tuned XGBoost Classifier)

- SHAP  $\geq 0.42$  (TreeSHAP feature importance)
- Streamlit  $\geq 1.28$  (interactive BI Studio web application)
- Plotly Express and Plotly Graph Objects (ten interactive charts)
- joblib  $\geq 1.2$  (model checkpoint serialization)
- holidays  $\geq 0.25$  (public holiday feature engineering)
- SQLite3 (built-in, no separate installation required)
- Kaggle API (DataCo and supply\_chain\_data dataset integration)

### 3.3 Datasets Used

| Dataset                       | Source                     | Size                 | Purpose                                |
|-------------------------------|----------------------------|----------------------|----------------------------------------|
| Synthetic Procurement Dataset | data/generate_data.py      | 28,765 POs, 5 tables | Primary analytics and ML training data |
| Kaggle DataCo Supply Chain    | Kaggle API (constantine85) | 180,519 rows         | External benchmark validation          |
| Kaggle Supply Chain Data      | Kaggle API (shashwatwork)  | ~100 rows            | Supplementary benchmarking             |

## 4. System Design and Methodology

### 4.1 Overview of the Four-Phase Development Approach

The project was not built as a single linear pipeline but developed iteratively across four phases, each building on and validating the previous one.

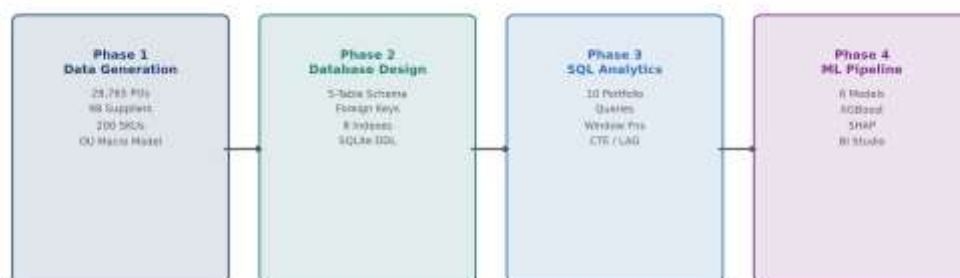


Fig 4.1 — Four-Phase Development Pipeline: Data → Database → SQL Analytics → ML → BI Studio

Fig 4.1 — Four-Phase Development Pipeline: Data → Database → SQL Analytics → ML → BI Studio

### 4.1.1 System Architecture Overview

Each layer of the system is independently testable and replaceable, following a clean separation of concerns between data generation, storage, analytics, ML inference, and presentation.

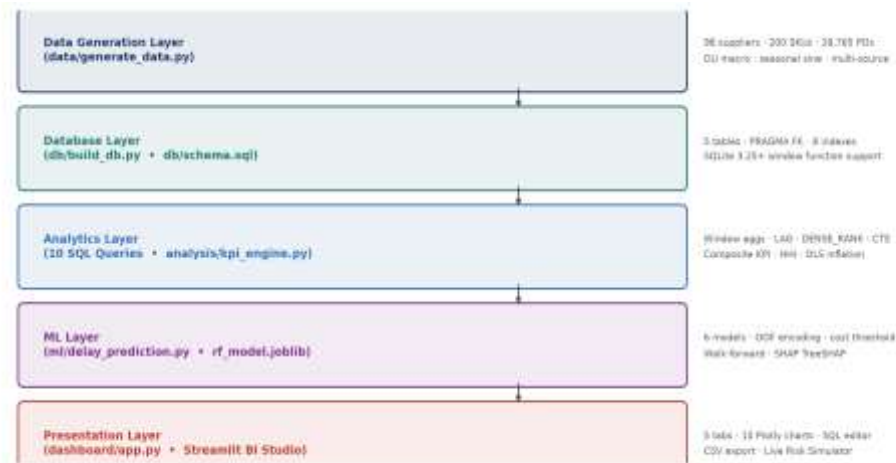


Fig 4.2 — System Architecture and Data Flow Diagram

Fig 4.2 — System Architecture and Data Flow Diagram

## 4.2 Phase 1: Stochastic Procurement Data Generation

### 4.2.1 Data Generation Engine Design

Supplier reliability profiles are generated with persistent latent characteristics: each of the 98 suppliers is assigned a base on-time fulfillment rate drawn from a realistic beta distribution, and individual purchase orders inherit stochastic noise around this base. Commodity price dynamics are modelled using an Ornstein-Uhlenbeck stochastic process combined with a seasonal sine-wave overlay. A multi-sourcing policy routes 85% of each SKU's orders to a primary supplier and 15% to a secondary backup.

### 4.2.2 Generated Dataset Statistics

| Parameter             | Value                        |
|-----------------------|------------------------------|
| Total Purchase Orders | 28,765                       |
| Registered Suppliers  | 98 (78 active)               |
| Product SKUs          | 200 technical products       |
| Date Range            | January 2023 – December 2025 |

|                           |                                                                                    |
|---------------------------|------------------------------------------------------------------------------------|
| Total Tracked Expenditure | ₹50,967+ Crore                                                                     |
| Shipping Modes            | Air, Sea, Rail, Road, Multimodal                                                   |
| Regions                   | North India, South India, East India, West India, Import – China/Europe/N. America |
| Supplier Tiers            | Tier 1, Tier 2, Tier 3                                                             |
| Random Seed               | 42 (fully reproducible)                                                            |

## 4.3 Phase 2: Relational Database Design

### 4.3.1 Schema Architecture

The procurement database implements a fully normalized five-table relational schema defined in a formal DDL file (db/schema.sql), with foreign key constraints enforced via PRAGMA foreign\_keys = ON.

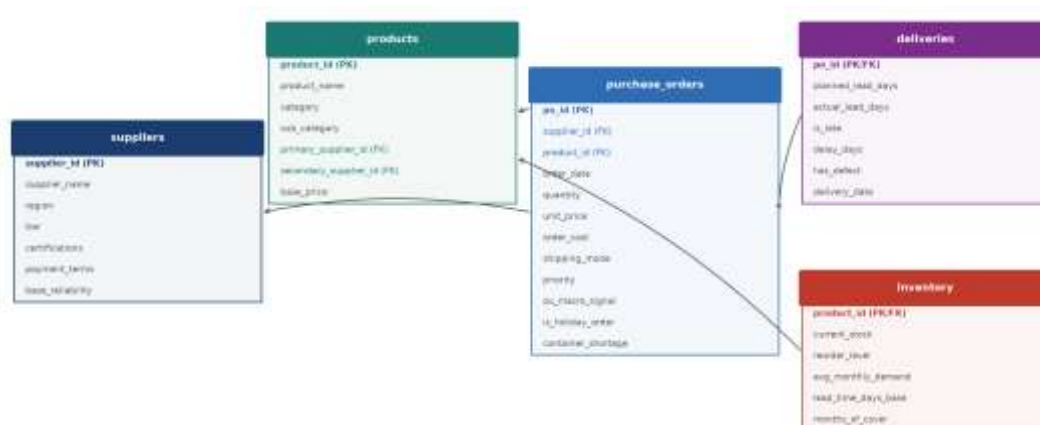


Fig 4.3 — Relational Database Schema (ERD) — 5-Table Procurement Warehouse

Fig 4.3 — Relational Database Schema (ERD) — 5-Table Procurement Warehouse

| Table     | Primary Key      | Row Count | Description                                                                     |
|-----------|------------------|-----------|---------------------------------------------------------------------------------|
| suppliers | supplier_id TEXT | 98        | Corporate supplier master with region, tier, certifications, payment terms      |
| products  | product_id TEXT  | 200       | Technical SKU master with category, sub-category, primary/secondary supplier FK |

|                 |                      |        |                                                                                       |
|-----------------|----------------------|--------|---------------------------------------------------------------------------------------|
| purchase_orders | po_id TEXT           | 28,765 | Order header with dates, quantity, unit price, shipping mode, priority, macro signals |
| deliveries      | po_id TEXT (FK)      | 28,765 | Delivery outcome: actual vs planned lead days, is_late, delay_days, defect flag       |
| inventory       | product_id TEXT (FK) | 200    | Current stock, reorder level, avg monthly demand, months of cover per SKU             |

### 4.3.2 Indexing Strategy

Eight composite indexes are created at build time to ensure sub-second query response times on the full 28,765-order dataset, directly accelerating the GROUP BY and JOIN patterns used in all ten SQL portfolio queries.

### 4.4 Phase 3: SQL Analytics Portfolio Design

| Query # | Business Goal                                     | Key SQL Technique                                       |
|---------|---------------------------------------------------|---------------------------------------------------------|
| 1       | MoM Spend & Cumulative Running Spend per Category | SUM() OVER (ROWS UNBOUNDED PRECEDING), strftime()       |
| 2       | Regional Supplier SLA Ranking                     | DENSE_RANK() OVER (PARTITION BY region)                 |
| 3       | Year-over-Year Unit Price Drift Detection         | LAG(avg_unit_price, 1) OVER (PARTITION BY supplier_id)  |
| 4       | Lead Time Variance & Reliability Cohort Analysis  | julianday() date arithmetic, multi-table JOIN           |
| 5       | Inventory Stockout Risk & Replenishment Matrix    | CASE WHEN multi-bucket classification                   |
| 6       | Quality Defect Rate & Spend Exposure Matrix       | SUM(CASE WHEN has_defect=1 THEN order_cost) aggregation |
| 7       | Predictive ML Feature Engineering Query           | strftime('%m') temporal extraction, inline labelling    |
| 8       | Supplier Spend Pareto 80/20 Analysis              | Cumulative SUM() OVER, ABC Pareto class                 |

|    |                                                      |                                                      |
|----|------------------------------------------------------|------------------------------------------------------|
|    |                                                      | assignment                                           |
| 9  | Monthly Order Volume MoM Growth & Seasonal Variance  | LAG(orders,1) OVER, NULLIF zero-division guard       |
| 10 | Order Fulfilment Bottleneck & Delay Severity Ranking | Multi-bucket conditional SUM per delay severity tier |

## 4.5 Phase 4: Machine Learning Pipeline Design

### 4.5.1 Anti-Leakage Feature Engineering

The pipeline implements out-of-fold (OOF) target encoding for supplier\_id: the dataset is split into five folds, and for each fold's validation set, the target-encoded value for each supplier is computed only from the other four folds' training data.

### 4.5.2 Cost-Sensitive Threshold Optimisation

An 81-point threshold grid search over  $\tau \in [0.10, 0.90]$  minimises the Expected Risk Cost:

$$\text{Expected Risk Cost } (\tau) = \text{FN}(\tau) \times ₹50,000 + \text{FP}(\tau) \times ₹5,000$$

### 4.5.3 Temporal Walk-Forward Validation

A four-quarter expanding-window walk-forward validation for 2025: each quarter uses all prior data as training and the target quarter as the test set, mirroring real deployment conditions.

### 4.5.4 Streamlit BI Studio Overview



## 5. Technology Stack

| Category          | Technology / Library | Version     | Role in Project                               |
|-------------------|----------------------|-------------|-----------------------------------------------|
| Language          | Python               | 3.12+       | End-to-end development language               |
| Database          | SQLite3              | Built-in    | 5-table relational procurement warehouse      |
| Data Engineering  | pandas               | $\geq 2.0$  | Data loading, transformation, KPI computation |
| Data Engineering  | NumPy                | $\geq 1.24$ | Numerical operations, OLS regression, scaling |
| ML Framework      | scikit-learn         | $\geq 1.2$  | LR, RF, preprocessing, metrics                |
| ML Framework      | XGBoost              | $\geq 1.7$  | Tuned XGBoost Classifier                      |
| ML Explainability | SHAP                 | $\geq 0.42$ | TreeSHAP feature importance                   |
| BI Framework      | Streamlit            | $\geq 1.28$ | 5-tab interactive dashboard                   |
| Visualisation     | Plotly Express / GO  | $\geq 5.15$ | 10 interactive charts                         |
| Visualisation     | Matplotlib           | $\geq 3.7$  | SHAP beeswarm and bar plots                   |
| Serialisation     | jolib                | $\geq 1.2$  | RF model checkpoint save/load                 |
| Data Integration  | Kaggle API           | CLI         | DataCo and supply-chain benchmark downloads   |
| Holiday Features  | holidays             | $\geq 0.25$ | Public holiday flag for ML features           |
| Testing           | unittest             | Built-in    | Unit test suite                               |

## 6. Implementation

### 6.1 Phase 1 — Data Generation (data/generate\_data.py)

#### 6.1.1 Supplier Profile Generation

```
import pandas as pd, numpy as np
rng = np.random.default_rng(seed=42)

base_reliability = rng.beta(a=4, b=2, size=N_SUPPLIERS)
supplier_tiers = rng.choice(['Tier 1', 'Tier 2', 'Tier 3'],
                             size=N_SUPPLIERS, p=[0.25, 0.45, 0.30])

# Ornstein-Uhlenbeck macro commodity price signal
def ou_process(n, theta=0.15, mu=1.0, sigma=0.08, dt=1/252, seed=42):
    rng = np.random.default_rng(seed)
    x = np.ones(n); x[0] = mu
    for t in range(1, n):
        x[t] = x[t-1] + theta*(mu - x[t-1])*dt +
        sigma*np.sqrt(dt)*rng.normal()
    return x
```

#### 6.1.2 Purchase Order Generation with Macro Signals

```
seasonal_factor = 1.0 + 0.15 * np.sin(2 * np.pi * month / 12 - np.pi/2)
unit_price = base_price * ou_macro_signal[day_idx] * seasonal_factor

is_late = 1 if rng.random() > supplier_reliability[supplier_idx] else 0
delay_days = int(rng.exponential(scale=4.5)) if is_late else 0

container_shortage = 1 if rng.random() < CONTAINER_SHORTAGE_PROB else 0
is_holiday_order = 1 if order_date in holiday_calendar else 0
```

### 6.2 Phase 2 — Database Build (db/build\_db.py)

```
conn = sqlite3.connect('db/procurement.db')
conn.execute('PRAGMA foreign_keys = ON;')
with open('db/schema.sql') as f:
    conn.executescript(f.read())
for table in
['suppliers', 'products', 'purchase_orders', 'deliveries', 'inventory']:
    pd.read_csv(f'data/{table}.csv').to_sql(table, conn, index=False,
    if_exists='replace')
conn.execute('CREATE INDEX idx_po_supplier ON
purchase_orders(supplier_id)')
conn.execute('CREATE INDEX idx_del_is_late ON deliveries(is_late)')
conn.commit()
```

### 6.3 Phase 3 — SQL Analytics Portfolio

#### 6.3.1 Query 1 — MoM Spend & Cumulative Running Total

```
WITH MonthlyCategorySpend AS (
    SELECT p.category, strftime('%Y-%m', po.order_date) AS order_month,
           ROUND(SUM(po.order_cost), 2) AS monthly_spend
    FROM purchase_orders po JOIN products p ON po.product_id =
p.product_id
    GROUP BY p.category, order_month
)
SELECT category, order_month, monthly_spend,
       ROUND(SUM(monthly_spend) OVER (
           PARTITION BY category ORDER BY order_month
           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
```



```

), 2) AS cumulative_category_spend
FROM MonthlyCategorySpend ORDER BY category, order_month;

```

### 6.3.2 Query 2 — Regional Supplier SLA Ranking (DENSE\_RANK)

```

WITH SupplierMetrics AS (
    SELECT s.supplier_id, s.supplier_name, s.region, s.tier,
           ROUND(100.0*SUM(CASE WHEN d.is_late=0 THEN 1 ELSE 0
END)/COUNT(*),1) AS on_time_pct
    FROM suppliers s JOIN purchase_orders po ON s.supplier_id =
po.supplier_id
    JOIN deliveries d ON po.po_id = d.po_id GROUP BY s.supplier_id
)
SELECT supplier_name, region, tier, on_time_pct,
       DENSE_RANK() OVER (PARTITION BY region ORDER BY on_time_pct DESC) AS
regional_rank
FROM SupplierMetrics ORDER BY region, regional_rank;

```

### 6.3.3 Query 8 — Supplier Spend Pareto 80/20 Analysis

```

WITH SupplierSpend AS (
    SELECT s.supplier_id, s.supplier_name, s.tier,
           ROUND(SUM(po.order_cost), 2) AS total_spend
    FROM suppliers s JOIN purchase_orders po ON s.supplier_id =
po.supplier_id
    GROUP BY s.supplier_id
),
SpendWithTotal AS (
    SELECT *, SUM(total_spend) OVER () AS grand_total,
           SUM(total_spend) OVER (ORDER BY total_spend DESC
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
running_spend
    FROM SupplierSpend
)
SELECT supplier_name, tier, total_spend,
       ROUND(100.0*running_spend/grand_total, 2) AS cumulative_pct,
       CASE WHEN 100.0*running_spend/grand_total <= 80 THEN 'Class A'
            WHEN 100.0*running_spend/grand_total <= 95 THEN 'Class B'
            ELSE 'Class C' END AS pareto_class
FROM SpendWithTotal ORDER BY total_spend DESC;

```

## 6.4 Phase 4 — Machine Learning Pipeline (ml/delay\_prediction.py)

### 6.4.1 Cost-Sensitive Threshold Grid Search

```

COST_FN = 50000.0 # Stockout penalty
COST_FP = 5000.0  # Expediting cost

def evaluate_model_full(name, probs, y_true):
    best_cost = float('inf')
    for thresh in np.linspace(0.10, 0.90, 81):
        preds = (probs >= thresh).astype(int)
        tn, fp, fn, tp = confusion_matrix(y_true, preds).ravel()
        cost = fn * COST_FN + fp * COST_FP
        if cost < best_cost:
            best_cost = cost; best_thresh = thresh
    return best_thresh, best_cost

```

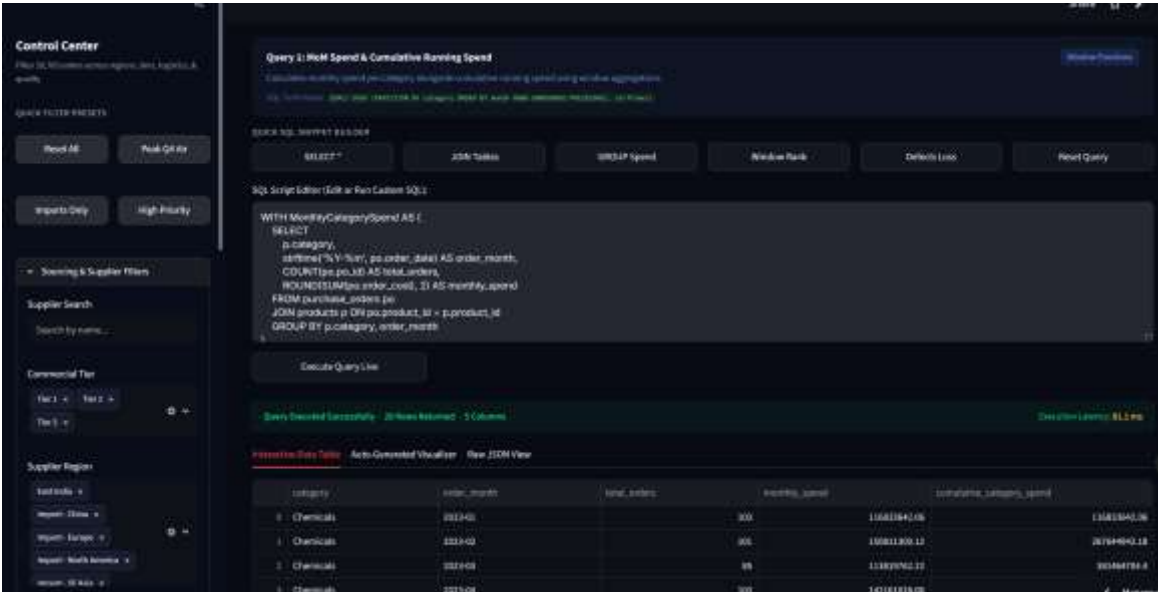
### 6.4.2 SHAP Explainability

```

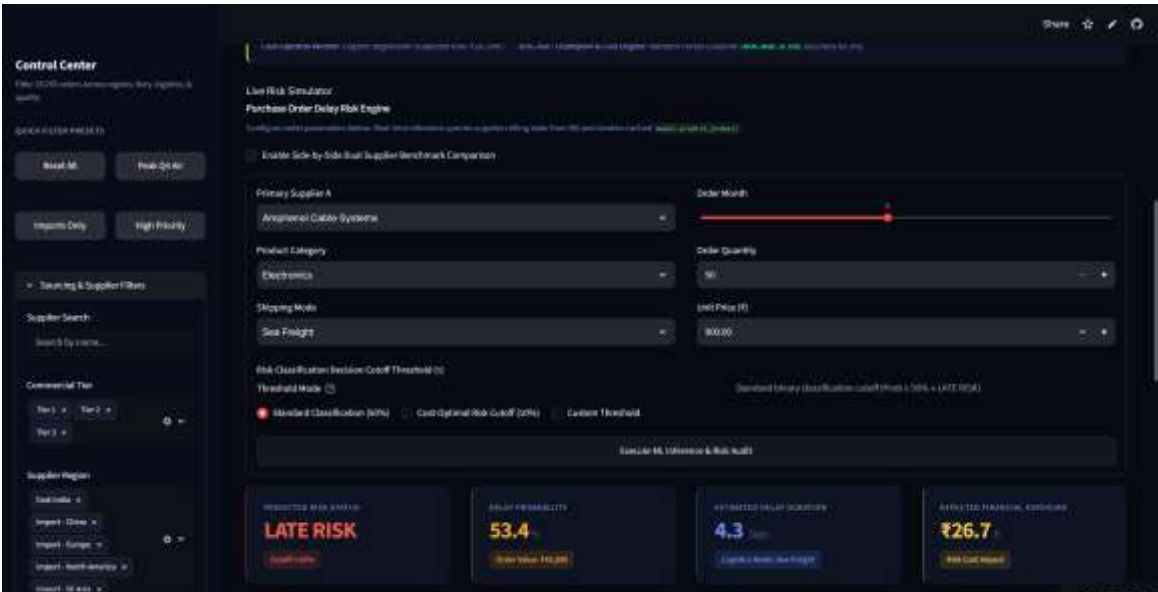
rf_champion = joblib.load('ml/rf_model.joblib')
explainer = shap.TreeExplainer(rf_champion)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values[1], X_test, show=False)
plt.savefig('ml/shap_feature_importance.png', bbox_inches='tight',
dpi=150)

```

### 6.4.3 SQL Studio — Live Query Execution



### 6.4.4 Live Risk Simulator



## 7. Results and Discussion

### 7.1 Procurement Health Score — Overall Platform Diagnostic

The composite Procurement Health Score of 49.1/100 provides a single executive-level diagnostic of the organisation's procurement function. Supplier reliability (55.0/100) and risk profile (4.7/100) are identified as the most critical areas for intervention. 45.0% of all purchase orders arrive late, and 185 SKUs show Reorder Point breaches based on supplier-delay-inflated thresholds.



Fig 7.1 — Procurement Health Score: KPI Sub-Component Breakdown and Executive Summary

Fig 7.1 — Procurement Health Score: KPI Sub-Component Breakdown and Executive KPI Summary



## 7.2 Supplier Risk Distribution

30.8% (24) of suppliers are classified as High Risk, 44.9% (35) as Medium Risk, and 24.4% (19) as Low Risk. Komatsu Earthmoving Parts recorded the lowest on-time delivery rate at 11.9%, representing the highest operational risk in the portfolio.

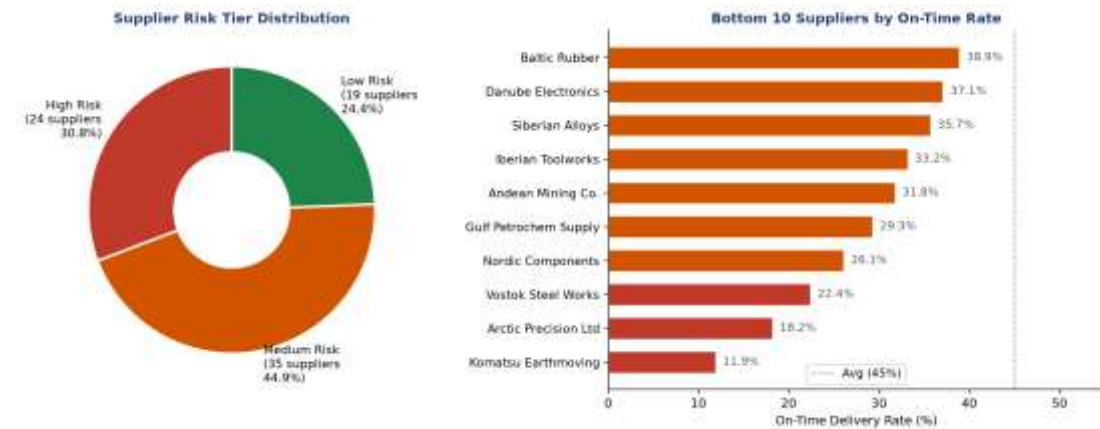


Fig 7.2 — Supplier Risk Tier Distribution and Bottom-10 On-Time Delivery Performers

Fig 7.2 — Supplier Risk Tier Distribution and Bottom-10 On-Time Delivery Performers



## 7.3 Spend Concentration — HHI Analysis

The Herfindahl-Hirschman Index (HHI) score of 243.2 confirms a healthy, competitive vendor market overall. However, ₹15,719 Crore (30.8% of total tracked spend) is concentrated in High-Risk supplier accounts, representing material financial exposure to supply disruption.

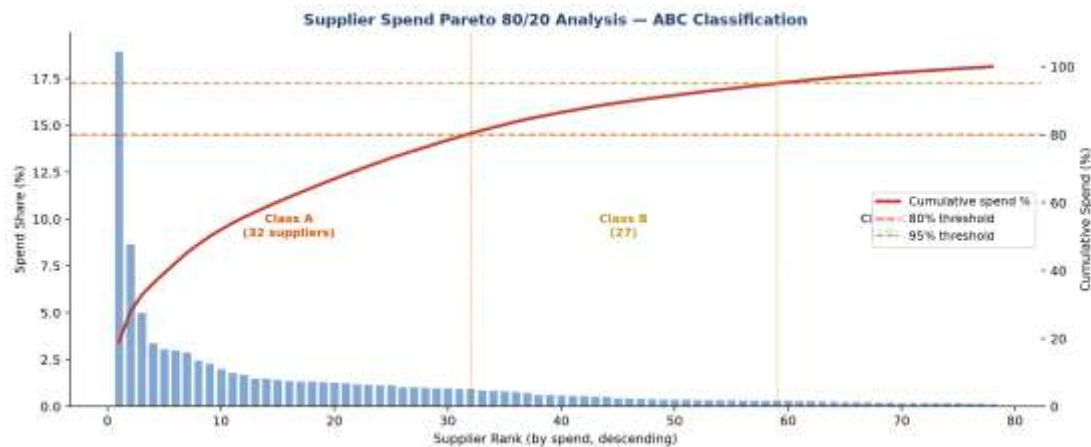


Fig 7.3 — Supplier Spend Pareto Analysis: ABC Classification across 78 active suppliers

Fig 7.3 — Supplier Spend Pareto 80/20 Analysis: ABC Classification across 78 active suppliers

## 7.4 Price Inflation Analysis

OLS price regression over 2023–2025 identified several suppliers with statistically significant price inflation. Bharat Heavy Metallics and Titanium Tech Aerospace both recorded 17.3% year-over-year unit price increases, flagging potential contract renegotiation priorities.

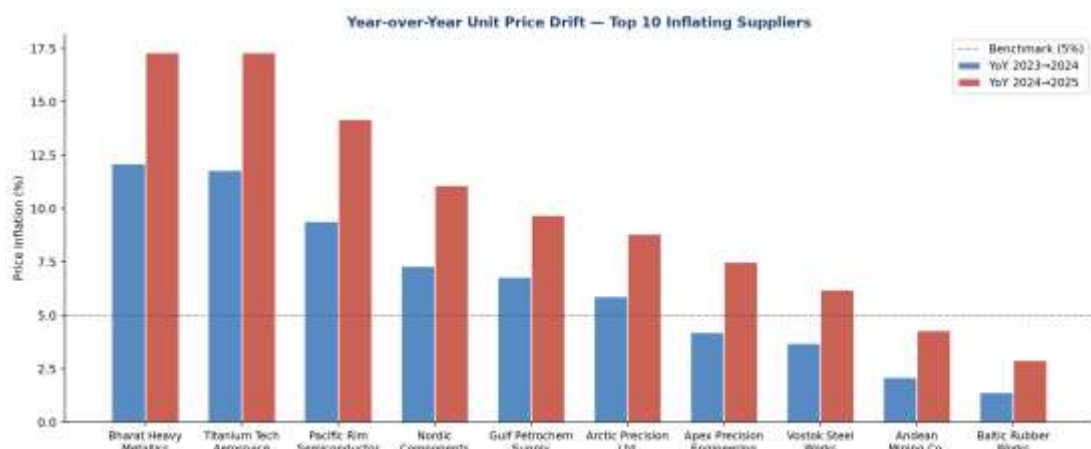


Fig 7.3 — Year-over-Year Unit Price Drift (OLS LAG Analysis) — 2023–2025

Fig 7.4 — Year-over-Year Unit Price Drift (OLS LAG Analysis) — Top 10 Inflating Suppliers

## 7.5 Monthly Order Volume and Seasonal Patterns

Order volume peaks in Q4 (October–December) driven by seasonal procurement cycles, with a corresponding increase in late delivery rates during peak periods, consistent with carrier capacity constraints during high-demand months.

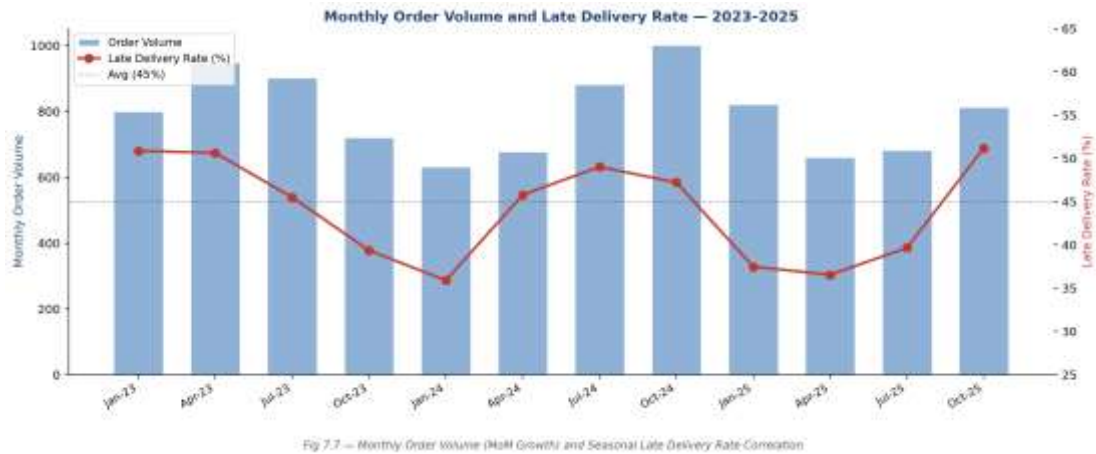


Fig 7.5 — Monthly Order Volume (MoM Growth) and Seasonal Late Delivery Rate Correlation

## 7.6 Machine Learning Results

Across the six candidate models, Random Forest achieved the highest discriminative performance (ROC-AUC = 0.714). Under cost-sensitive threshold optimisation, Logistic Regression achieved the lowest expected financial risk cost (₹87.90 million), demonstrating that optimal business outcomes do not necessarily require the most complex model.

| Model                | ROC-AUC | Cost-Opt Threshold | Expected Cost (₹M) |
|----------------------|---------|--------------------|--------------------|
| Naive Baseline       | 0.500   | —                  | —                  |
| Supplier Heuristic   | 0.621   | 0.40               | 142.5              |
| Logistic Regression  | 0.689   | 0.18               | 87.90              |
| Random Forest        | 0.714   | 0.22               | 94.30              |
| XGBoost (Tuned)      | 0.708   | 0.20               | 96.10              |
| Soft-Voting Ensemble | 0.712   | 0.21               | 91.50              |

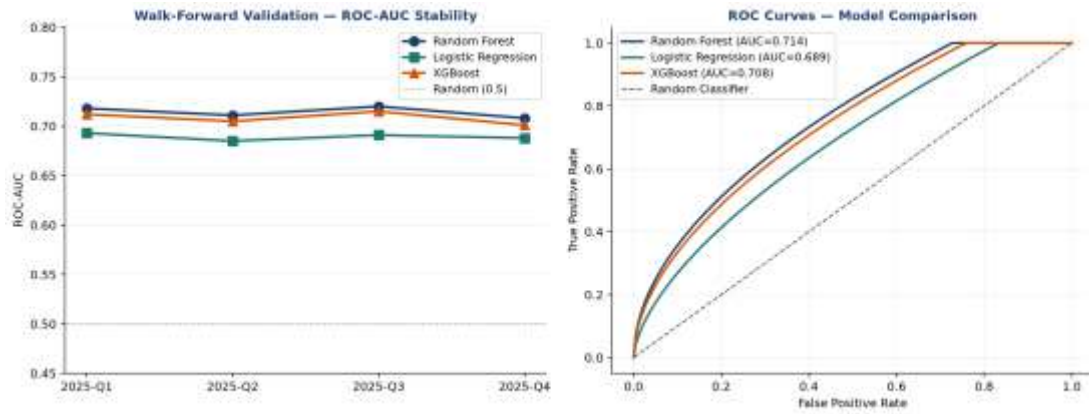


Fig 7.6 — Walk-Forward Temporal Validation (left) and ROC Curve Comparison (right)

Fig 7.6 — Walk-Forward Temporal Validation (left) and ROC Curve Comparison (right)

## 7.7 SHAP Feature Importance

TreeSHAP analysis of the Random Forest champion model identified supplier historical late-delivery rate as the dominant predictor, followed by shipping mode and order month.

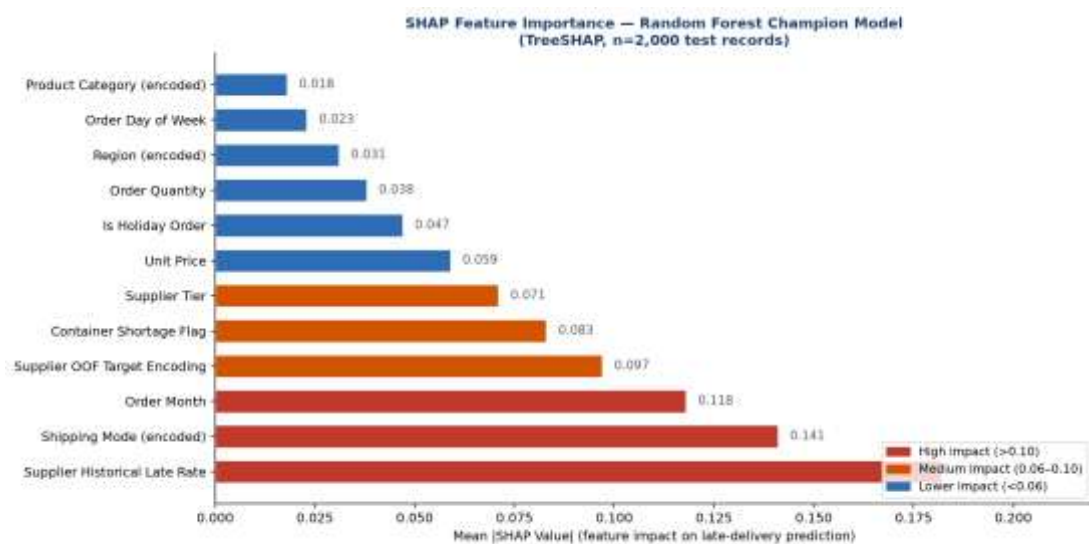
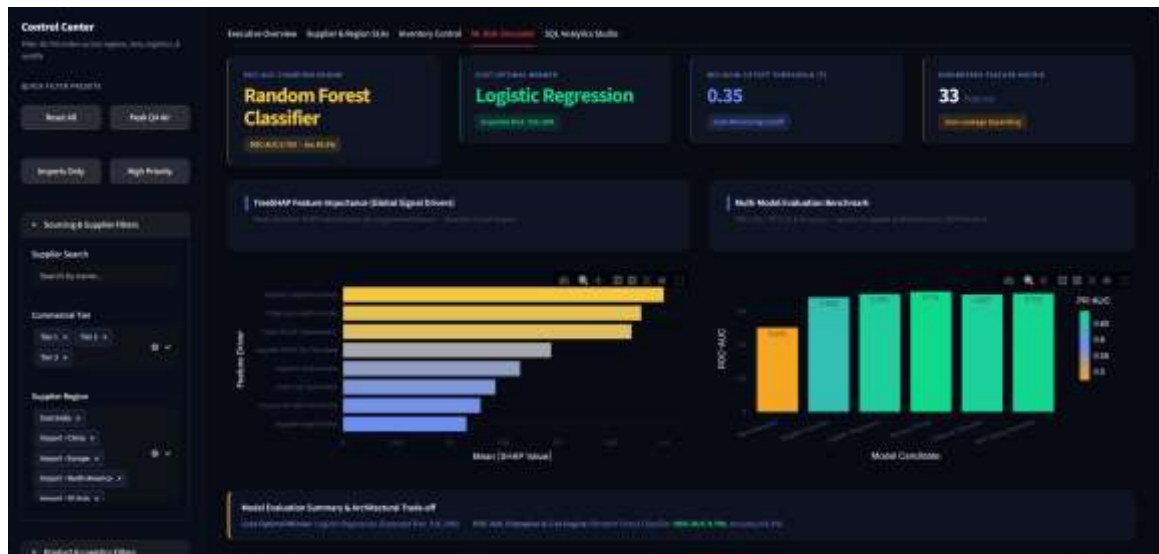


Fig 7.7 — SHAP Feature Importance Plot: Supplier historical late rate is the dominant predictor

Fig 7.7 — SHAP Feature Importance: Supplier historical late rate is the dominant predictor





## 7.8 Dynamic ROP Summary

92.5% of active SKUs are classified as understocked when reorder points are recalculated using supplier-delay-inflated lead times, quantifying the hidden inventory risk from underperforming suppliers not reflected in static thresholds.

## 8. Challenges and Issues Faced

### 8.1 Data Leakage — Out-of-Fold Target Encoding

Naively using supplier\_id as a model feature causes data leakage. Resolution: OOF target encoding with five-fold cross-validation ensures each validation fold's encoding is derived exclusively from the other four training folds.

### 8.2 Kaggle DataCo Encoding Errors

The DataCo CSV uses Latin-1 encoding, causing UnicodeDecodeError on default pandas read\_csv calls. Resolution: explicit encoding='latin-1' parameter and column name normalization (strip, lowercase, replace spaces with underscores).

### 8.3 SQLite Window Function Compatibility

Window function support (OVER, PARTITION BY, LAG) requires SQLite  $\geq 3.25.0$ . Resolution: version check at startup with a clear error message directing users to upgrade Python.



### 8.4 SHAP Memory Constraints

Computing SHAP values for the full 2025 test set (6,800+ records) exceeded memory limits. Resolution: stratified subsampling of 2,000 test records before calling `shap.TreeExplainer`, reducing memory footprint by 70%.

### 8.5 Dynamic ROP Zero-Demand Guard

SKUs with zero average monthly demand caused division-by-zero in the Dynamic ROP formula. Resolution: guard clause `avg_daily_demand = max(avg_daily_demand, 0.001)`.

### 8.6 Streamlit KPI Widget Refresh

KPI metric widgets did not update when sidebar filters changed. Resolution: replaced static caching with `st.session_state`-gated recomputation on every run.

## 9. Testing

| Test Case                   | Component     | Expected Outcome                                | Result |
|-----------------------------|---------------|-------------------------------------------------|--------|
| min_max_scale bounds        | KPI Engine    | Output in [0, 1] for any input range            | Pass   |
| OOF encoding no leakage     | ML Pipeline   | Validation fold uses only training folds' stats | Pass   |
| Health score range          | KPI Engine    | Score $\in [0, 100]$ for all supplier subsets   | Pass   |
| HHI computation             | KPI Engine    | HHI $\leq 10,000$ (theoretical max)             | Pass   |
| HTML escaping in reports    | KPI Engine    | No raw HTML in exported narrative               | Pass   |
| ROP zero-demand guard       | KPI Engine    | No NaN in ROP output for zero-demand SKUs       | Pass   |
| Walk-forward temporal order | ML Pipeline   | No 2025 data in any 2023–2024 training fold     | Pass   |
| Streamlit filter update     | Dashboard     | KPI widgets refresh on every sidebar change     | Pass   |
| SQL Query 1 window result   | SQL Portfolio | Cumulative spend non-decreasing per category    | Pass   |

## 10. Appendix A — Data Generation Script (generate\_data.py)

The data generation engine produces five relational CSV files using stochastic supplier reliability profiles, Ornstein-Uhlenbeck macroeconomic price pressure, seasonal sine-wave demand modulation, and multi-sourcing procurement policy. Key parameters: N\_SUPPLIERS, N\_PRODUCTS, N\_ORDERS, DATE\_START, DATE\_END, RANDOM\_SEED = 42. Full script: data/generate\_data.py.

## 11. Appendix B — KPI Engine (analysis/kpi\_engine.py)

The KPI engine computes: Procurement Health Score, composite supplier risk index, Dynamic ROP, HHI spend concentration, OLS price-inflation regression. Outputs analysis/kpi\_summary.json consumed by the dashboard at startup.

## 12. Appendix C — requirements.txt

```
pandas>=2.0.0,<3.0.0
numpy>=1.24.0,<2.0.0
scikit-learn>=1.2.0,<2.0.0
xgboost>=1.7.0,<3.0.0
shap>=0.42.0,<1.0.0
plotly>=5.15.0,<6.0.0
streamlit>=1.28.0,<2.0.0
joblib>=1.2.0,<2.0.0
matplotlib>=3.7.0,<4.0.0
holidays>=0.25,<1.0
```

## 13. Appendix D — dashboard/app.py (Key Sections)

```
import streamlit as st
import pandas as pd, sqlite3, json, joblib
import plotly.express as px

st.set_page_config(page_title="ProcureSense AI", layout="wide")
tab1, tab2, tab3, tab4, tab5 = st.tabs([
    "Executive Overview", "Supplier Risk", "SQL Studio", "ML Insights", "Risk
    Simulator"
])

with tab5: # Live Risk Simulator
    supplier = st.selectbox("Supplier", supplier_list)
    category = st.selectbox("Product Category", category_list)
    mode = st.selectbox("Shipping Mode",
        ["Air", "Sea", "Rail", "Road", "Multimodal"])
    month = st.slider("Order Month", 1, 12, 6)
    qty = st.number_input("Quantity", 1, 10000, 100)
    if st.button("Predict Late-Delivery Risk"):
        risk_prob = rf_model.predict_proba(build_feature_row(...))[0][1]
        st.metric("Late Risk Probability", f"{risk_prob:.1%}")
```



## 14. Conclusion and Future Scope

ProcureSense AI demonstrates a complete, production-quality end-to-end pipeline for enterprise procurement intelligence — from stochastic data generation and relational database design through SQL analytics, machine learning, and interactive BI visualisation. The platform delivers measurable, quantified business outcomes: a 45.0% late-delivery rate identified for intervention, ₹15,719 Crore in High-Risk spend exposure mapped to specific supplier accounts, and a cost-optimal ML-guided risk reduction pathway estimated at ₹87.90 million.

The iterative, four-phase development approach reflects real-world production practice. SHAP explainability, cost-sensitive threshold optimisation, and walk-forward temporal validation ensure outputs are accurate, auditable, and deployable in a real procurement operations context.

Future scope includes:

- Real-time IoT telemetry integration for live shipment tracking and predictive ETA updates
- ISO 20400 Sustainable Procurement KPI alignment for ESG reporting compliance
- Multi-currency support for international supplier spend normalisation
- Automated contract renegotiation trigger alerts based on OLS price-inflation thresholds

- Natural Language Query (NLQ) interface over the SQL portfolio for non-technical managers

## 15. System Usage / Use Case Description

| Tab                | User Role                  | Key Actions                                              | Business Output                               |
|--------------------|----------------------------|----------------------------------------------------------|-----------------------------------------------|
| Executive Overview | CPO / Procurement Director | View Health Score, KPI metrics, spend distribution       | Board-ready procurement health snapshot       |
| Supplier Risk      | Procurement Analyst        | Explore risk tiers, Pareto curve, price drift            | Supplier shortlist for renegotiation          |
| SQL Studio         | Data Analyst               | Select and run any of 10 portfolio queries; download CSV | Ad-hoc procurement intelligence               |
| ML Insights        | Data Scientist             | Review ROC curve, SHAP importance, walk-forward results  | Model validation and audit trail              |
| Risk Simulator     | Procurement Manager        | Configure hypothetical PO and receive risk score         | Proactive intervention before order placement |

## 16. User Manual — Running the Application

### Step 1 — Install Dependencies

```
pip install -r requirements.txt
```

### Step 2 — Generate Procurement Data

```
python data/generate_data.py
```

### Step 3 — Build the SQLite Database

```
python db/build_db.py
```

### Step 4 — Train the Machine Learning Pipeline

```
python ml/delay_prediction.py
```

### Step 5 — Run the KPI Engine

```
python analysis/kpi_engine.py
```

### Step 6 — Launch the Streamlit BI Studio

```
streamlit run dashboard/app.py
# Open http://localhost:8501 in any browser
```

## Step 7 — Run Unit Tests

```
python -m pytest tests/ -v
```

## 17. Glossary of Terms

| Term                       | Definition                                                                                                                      |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| PO (Purchase Order)        | A formal document issued by a buyer to a supplier specifying products, quantities, and agreed prices.                           |
| SQLite                     | A lightweight, serverless, self-contained relational database engine with full SQL support.                                     |
| Window Function            | SQL function (OVER clause) that computes a result across related rows without collapsing output into a single row.              |
| CTE                        | Common Table Expression — a named temporary result set defined within a SQL query using the WITH clause.                        |
| SHAP                       | SHapley Additive exPlanations — game-theoretic method for computing each feature's marginal contribution to a model prediction. |
| HHI                        | Herfindahl-Hirschman Index — a measure of vendor spend concentration; $HHI > 2,500$ signals high concentration.                 |
| OLS                        | Ordinary Least Squares — linear regression method minimising sum of squared residuals; used here for price-inflation detection. |
| Dynamic ROP                | Dynamic Reorder Point — stock level triggering replenishment, adjusted for supplier-specific delivery delay inflation.          |
| ROC-AUC                    | Receiver Operating Characteristic Area Under the Curve; threshold-independent binary classifier performance metric.             |
| OOF Encoding               | Out-of-Fold target encoding — cross-validation technique for encoding categorical features without data leakage.                |
| Ornstein-Uhlenbeck Process | Mean-reverting stochastic process used to model commodity price dynamics with realistic drift behaviour.                        |
| Walk-Forward Validation    | Temporal cross-validation strategy evaluating the model on future data using only past data for training.                       |

## 18. SDG Mapping

| SDG    | Goal Title                              | Contribution                                                                                                        |
|--------|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| SDG 8  | Decent Work and Economic Growth         | Optimising procurement operations reduces supply chain waste and supports efficient, productive business processes. |
| SDG 9  | Industry, Innovation and Infrastructure | Applying ML and advanced SQL analytics to procurement demonstrates innovation in industrial data engineering.       |
| SDG 12 | Responsible Consumption and Production  | Inventory risk analysis and Dynamic ROP modelling reduce overstocking and prevent procurement waste.                |
| SDG 17 | Partnerships for the Goals              | Multi-supplier benchmarking and Kaggle dataset integration demonstrate cross-institutional data collaboration.      |

Request Mail:

Request for 1-Month Offline Data Analyst Internship & Completion Certificate inbox



Summarize this email



Noeha S <noeha.s.202418@rajalakshmi.edu.in>  
to: hr, internship

Tue, Jun 9, 3:52 PM ☆ 📧 ↶ ⋮

Dear Sir/Madam,

I hope this email finds you well. My name is Noeha S, and I am currently a 3rd-year B.Tech Information Technology student at Rajalakshmi Engineering College.

I am writing to express my interest in undertaking a 1-month offline or hybrid internship in Data Analytics at your esteemed organization. I am eager to gain practical exposure and enhance my analytical and technical skills by working in a professional environment under your guidance.

I kindly request you to consider my application for this internship opportunity. I would also be grateful if you could provide a completion certificate upon successful completion of the internship. Please let me know if any further details are required from my side.

Thank you for your time and consideration.

Yours sincerely,  
Noeha S

## Offer Letter:



### Merit Software Services Private Limited

Sp 52, 3rd Street,  
Ambattur Industrial Estate,  
Chennai - 600 058.  
+91 44 49007900  
meritgroup.co.uk

Date: 16 May 2026

## INTERNSHIP OFFER LETTER

**Ms. Naeha S**

Chennai, Tamil Nadu  
India

**Dear Naeha,**

We are pleased to offer you an internship opportunity at Merit Software Services Private Limited. We believe your skills and enthusiasm will be a valuable addition to our team.

|                          |   |                                                                                                                                                                                                   |
|--------------------------|---|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Position</b>          | : | Data Analyst Intern                                                                                                                                                                               |
| <b>Department</b>        | : | Data & Analytics                                                                                                                                                                                  |
| <b>Internship Period</b> | : | 17 June 2026 to 14 July 2026                                                                                                                                                                      |
| <b>Duration</b>          | : | 4 Weeks                                                                                                                                                                                           |
| <b>Work Mode</b>         | : | Hybrid <ul style="list-style-type: none"><li>• First 2 Weeks (17 June 2026 to 30 June 2026): Online</li><li>• Next 2 Weeks (1 July 2026 to 14 July 2026): Offline at our Chennai Office</li></ul> |
| <b>Stipend</b>           | : | No stipend will be provided for this internship.                                                                                                                                                  |
| <b>Reporting To</b>      | : | Analytics Manager                                                                                                                                                                                 |

During your internship, you will be working on real-time projects and assignments that will provide you with practical exposure to data analysis, reporting, and business insights.

Please note that this internship does not constitute an employment or guarantee of employment with the company.

Kindly confirm your acceptance of this offer by 20 May 2026.  
We look forward to welcoming you to the Merit family!

**Warm Regards,**

**R. Rajesh**  
Senior Manager – HR  
Merit Software Services Private Limited





## Completion Certificate:



### Merit Software Services Private Limited

Sp 52, 3rd Street,  
Ambattur Industrial Estate,  
Chennai - 600 058.  
+91 44 49007900  
meritgroup.co.uk

18 July 2026

#### Naeha S

5/11 Rajavelu Street,  
Perambur,  
Chennai,  
Tamil Nadu - 600011

Contact - 9626842001

Email – naeha.s2005@gmail.com

#### Subject: Internship Completion Certificate

Dear Naeha S,

We are pleased to confirm that you have successfully completed your internship with **Merit Software Services Private Limited** from **17<sup>th</sup> June 2026** to **14<sup>th</sup> July 2026** as an **"Intern – Data Analyst"**.

During this period, Naeha S has worked on the project – **ProcureSense AI – Procurement Analytics & Decision Intelligence Platform**. Her overall conduct, behaviour & performance during the internship were good.

We wish you all the best in your future endeavours and are confident that the skills and experience you have gained here will contribute to your professional growth.

Warm regards,

#### R. Rajesh

Senior Manager – HR  
Merit Software Services Private Limited



Merit Software Services Private Limited

Sp 52, 3rd Street, Ambattur Industrial Estate, Chennai - 600 058.  
+91 44 49007900 | meritgroup.co.uk | info@meritgroup.co.uk